Kenyan Campus Student Support and Outcome Modelling: An Advanced Python Project

KCA University

Bachelor of Science in Data Science

CPP 4201 - Advanced Python Programming

Group members

Tony Kenga - 24/03652

Baraka Blessings - 24/04272

Academic project report

21 October 2026

Abstract

This project develops a reproducible Python workflow for extracting, validating and analysing the UCI Machine Learning Repository dataset *Predict Students' Dropout and Academic Success*. Its central aim is to show how advanced Python methods support a complete data-science process, not merely to maximize a classifier score. The observed outcome is treated as a three-class target: Dropout, Enrolled and Graduate.

The inspected file has 4,424 records, 36 predictors and one target. A defensive validation stage checks format, row widths, schema, missingness, duplicate observations and target labels. A stratified 80/20 split reserves 885 records for final evaluation. Three scikit-learn pipelines compare Logistic Regression, Random Forest and Gradient Boosting using five-fold stratified cross-validation on training data. The selected Logistic Regression model achieved mean training cross-validation macro-F1 of 0.705 (standard deviation 0.030), then held-out macro-F1 of 0.716 and accuracy of 0.756.

The project now also includes a separate Streamlit demonstration for Kenyan campus support. Its rule-based chatbot routes academic and CAT pressure, financial stress, anxiety, loneliness, low mood and immediate-safety language to fixed responses and Kenyan signposting. It is not connected to model predictions, does not diagnose, and does not send student text to a generative AI service. This distinction lets the project demonstrate student-centred support without presenting retrospective academic labels as wellbeing measurements.

The code demonstrates typed modular functions, dataclasses, a Protocol, an Enum, context managers, a lazy generator, bounded retry decoration, pattern matching, logging, custom exceptions and pytest. Semester performance features rank highest in permutation analysis; they also restrict any interpretation as an early warning. Predictions describe recorded outcomes, not causes or guaranteed future behaviour.

Keywords: Kenyan campus, student support, intent classification, Python, ETL, reproducibility

1. Introduction

Higher education institutions collect demographic, academic and socioeconomic information. A carefully scoped analysis can describe how this information relates to recorded student outcomes. This project uses a public UCI dataset to compare three familiar multiclass classifiers while making the surrounding Python workflow visible and testable.

The work is framed as an Advanced Python Programming project. Its main question is how modular software design, explicit contracts, validation, automated checks and reproducible execution help make an analytical result more trustworthy and explainable. Predictive performance is necessary for a useful experiment, but is not the only assessment criterion.

The UCI target records whether a student dropped out, remained enrolled or graduated. The three labels are retained because collapsing them would change the problem and discard a meaningful observed class. A model that performs well on this dataset still cannot establish why a student left, nor prove that an intervention will change an outcome.

The modelling study is intentionally limited to one tabular dataset, three estimators, modest fixed settings and one held-out evaluation. A local Streamlit web interface provides a separate student-support demonstration, but it does not deploy online or connect support chat to outcome predictions. This separation is a core architectural and ethical boundary.

2. Problem, objectives and research questions

The concrete problem is to estimate the dataset's observed `Target` class from its 36 available predictors using a reproducible Python pipeline. The labels are `Dropout`, `Enrolled` and `Graduate`.

The objectives are to: (1) obtain and inspect the official dataset; (2) validate structural and value assumptions; (3) distinguish nominal integer codes from numeric measurements; (4) compare three classifiers under the same stratified protocol; (5) produce transparent evaluation and importance artifacts; (6) localize a support-chat prototype to Kenyan campus concerns; and (7) test the pipeline and intent routing.

The research questions are: How effectively can the available features predict the observed outcome? Which features contribute most to the selected model's predictive performance? How can deterministic intent rules present practical, locally relevant signposting for common campus challenges? How do advanced Python techniques improve reliability, readability, modularity, testing and reproducibility?

The third question is central. For example, a data validator gives failures a clear meaning; a scikit-learn pipeline keeps preprocessing attached to the estimator; immutable configuration records the experiment choices; and automated tests check both normal behaviour and invalid inputs. These choices reduce the chance that a result depends on undocumented manual steps.

The scope is one focused assignment. Results are descriptive of this dataset and split. They do not claim a causal mechanism, a universally generalizable risk score, or an operational tool for decisions about individual students.

3. Background and related work

Educational data mining uses institutional records to study student performance, progression and attrition. The source dataset was assembled from demographic, socioeconomic, macroeconomic and academic information at a Portuguese higher education institution. It contains academic progress from the first and second semesters, which is important for interpreting its predictive setting.

Realinho and colleagues describe a multi-class phased prediction approach for higher-education academic performance and dropout. Their work is closely related to the present outcome categories and source data. The dataset authors also publish a companion dataset description in *Data*. The current project uses their source as a benchmark for a software-engineering exercise; it does not attempt to reproduce their full research design.

Classification metrics answer different questions. Accuracy is the proportion of all predictions that are correct. Macro-F1 computes precision and recall balance separately for each class and gives the class scores equal weight. With different class frequencies, these metrics can lead to different model rankings. Therefore, macro-F1 is primary here and accuracy is secondary.

Permutation importance estimates the score decrease after randomly shuffling a feature. It can be used with a fitted model without reading its internal coefficients. Its result depends on the chosen metric, data and model; correlated variables can share predictive information. It is not a causal explanation.

4. Software and methodological background

Python supports a pipeline through composable functions and objects. Context managers bind resource acquisition and cleanup to a block. Type hints make expected values visible to readers and static analysis. Dataclasses are useful for compact structured configuration. Protocols describe the methods an interchangeable stage should expose without imposing inheritance.

Scikit-learn's `Pipeline` composes preprocessing and estimation, while `ColumnTransformer` applies different transformations by named columns. When these objects are included in cross-validation, each fold fits preprocessing using only its training portion. This prevents a scaler or encoder from learning from validation examples before scoring.

The experiment first separates a stratified held-out partition, then compares candidates using cross-validation inside the training data. Only the highest mean CV macro-F1 candidate is evaluated on the holdout. This is a simple model-selection procedure, not a large hyperparameter search. The selected model is refitted to the complete training partition before final evaluation.

Automated tests complement scientific checks by making expected behaviour repeatable. A unit test cannot prove the research design is appropriate, but it can detect accidental schema changes, missed validation, broken retries or missing report artifacts. This is why both methodology documentation and software tests are part of the deliverable.

5. Dataset source and provenance

The dataset is *Predict Students' Dropout and Academic Success*, UCI Machine Learning Repository dataset ID 697. The official ZIP was downloaded on 21 October 2026 from UCI's static public archive endpoint. The dataset is licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). The corresponding UCI dataset citation and the article describing the data are listed in the references.

The ZIP archive contained exactly one relevant member named `data.csv`. It is 533,230 bytes after extraction, encoded as UTF-8 with a byte-order mark and separated by semicolons. The extracted CSV SHA-256 is `3ef126de5cefff26eb11fbb4237f1a1401cb64b488e2f1d598c23cedeb4c45ae`. The ZIP SHA-256 is `e90e55fd65ec462ae283ebeb2cca409319e3460ed898d8754f62fb35cc83a65d`.

The observed dimensions are 4,424 rows and 37 columns: 36 inputs and one `Target`. The three target counts are Graduate 2,209, Dropout 1,421 and Enrolled 794. These are observed file facts, not assumptions copied from a reference description. The CSV and archive are retained locally but excluded from version control by `.gitignore`.

6. Data quality and exploratory inspection

Inspection used the ZIP member listing and direct parsing of the delimited file. Each nonblank data row had 37 fields. Column names were unique. No malformed row widths, missing or blank cells, duplicate rows or duplicate columns were detected. All predictor columns parsed as numeric values. This agrees with the source's reported preprocessed status; the project does not invent missing-value issues.

Two source-label quirks were retained: the column is spelled `Nacionality`, and `Daytime/evening attendance` has a trailing tab in its header. Preserving the source schema avoids silently changing fields and makes data lineage easier to check. The original feature names also remain present in importance outputs.

The class distribution is imbalanced: Graduate accounts for about 49.9%, Dropout 32.1%, and Enrolled 17.9%. A majority-class prediction would already have about 49.9% accuracy, illustrating why accuracy alone is insufficient. Macro-F1 assigns the minority Enrolled class equal influence in the comparison.

The validator checks file existence, row width, expected column count, unique columns, target column location, missingness, target labels and numeric predictors. A repeated row would be logged and retained rather than silently removed. On this actual source file, that rule reported no duplicates.

7. Feature meaning and preparation

The 36 predictors mix nominal code values, binary indicators, counts and continuous measurements. Course, application mode/order, nationality, qualification and occupation codes are treated as categories rather than as magnitudes. Binary attributes such as gender, debtor and scholarship holder are also passed through the nominal encoder.

Grades, age, macroeconomic values and curricular-unit counts are treated as numeric measurements. Numeric columns are standardized. The one-hot encoder handles categories not seen in a training fold by ignoring those columns in the transformed representation. The full transformer remains within an sklearn pipeline.

This division is a modelling choice based on feature meanings, not merely on the fact that the source values are integers. An integer code for a course does not mean course 20 is twice course 10. Conversely, counts have a meaningful quantitative ordering. Tree models are invariant to monotonic scaling in their split logic, but sharing the transformer design keeps the comparison workflow consistent.

The target is separated from predictors before the split. No observations are removed and no imputation is performed because no missing cells were observed. Potential deployment risks are distinguished from confirmed anomalies: the available data is clean structurally, yet timing and external validity remain concerns.

8. System architecture and Kenyan campus interface

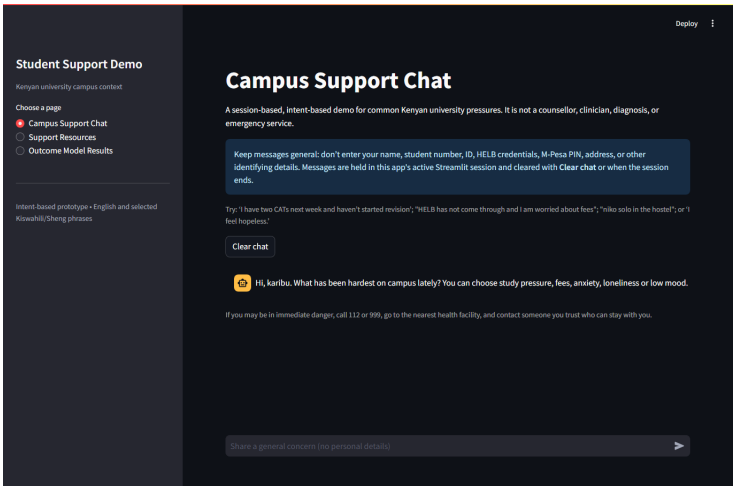
The implementation is an installable `src`-layout package. `extract.py` downloads and safely extracts the UCI archive. `validate.py` reads the semicolon CSV and enforces schema and value rules. `transform.py` separates predictors and outcome and builds a column-wise preprocessing transformer. `model.py` defines the three candidates and performs training-only cross-validation. `evaluate.py` reports final metrics and figures. `cli.py` orchestrates the stages and saves run metadata. The separate `support_chatbot.py` contains intent patterns and fixed responses; `support_app.py` presents chat, resource and outcome-results pages using Streamlit.

Each module has a narrow role. Custom exceptions distinguish download failure, invalid archive, schema mismatch and row validation problems. Logging records progress and warnings. Frozen dataclasses make experiment settings and result records explicit. A small `FrameValidator` Protocol expresses a replaceable validation contract without requiring a framework. The support app is a distinct presentation layer; the rule classifier returns a `SupportReply` containing intent, message and resource keys. Streamlit keeps messages in its active session state and offers a clear-chat control.

The command-line program accepts data/output paths, a random seed and a `--download` flag. The optional Streamlit interface is started separately with `python -m streamlit run src/dropout_pipeline/support_app.py`. Data, model comparison results and configuration are saved in separate files. The results page reads these artifacts; the chatbot never consumes a student's predicted outcome.

The coursework does not require a database or cloud deployment. The local app needs Streamlit; the original ETL/model CLI remains independently usable. A text flow is: student message -> ordered regular-expression intent matcher -> fixed safe reply -> optional verified resource links. The model flow remains: UCI archive -> extraction -> validation -> transformation pipeline -> cross-validated model selection -> held-out evaluation -> JSON/PNG artifacts -> results page.

Running interface: chatbot welcome and privacy notice



9. ETL validation and failure handling

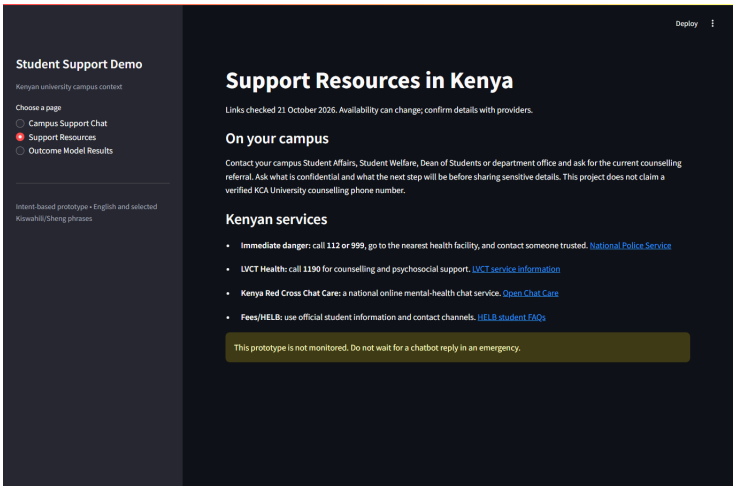
The `download_dataset` function uses a bounded retry decorator around the HTTP fetch. It retries network-related errors up to three attempts with increasing brief delays and then raises a domain-specific `DownloadError`. This is appropriate for transient network faults, not for repeatedly retrying a malformed dataset.

The downloaded bytes are opened as a ZIP through a context manager. The extraction code checks that exactly one `data.csv` member exists before writing it. SHA-256 is calculated by reading the file in chunks, which avoids loading a potentially large file into memory merely to hash it.

Validation is defensive but specific. It checks row widths using a lazy line generator, then lets pandas parse the semicolon-separated data. It rejects unexpected target labels, missing values, duplicate column names and a nonnumeric predictor. Failures report actionable context through the exception hierarchy.

The CSV and archive are not overwritten with cleaned variants. If a future version of the source contains an anomaly, the validator will fail or log it; the group can decide and document an explicit policy rather than allowing an implicit row deletion to alter the experiment.

Running interface: Kenyan support resources and campus referral route



10. Advanced Python concepts I: types, data structures and interfaces

Type hints document inputs and outputs such as `Path`, pandas `DataFrame`, `Series`, numpy arrays and model-result records. The configured mypy check aims to catch inconsistent use before a full run. Type hints do not validate external CSV contents at runtime, so explicit data checks remain necessary.

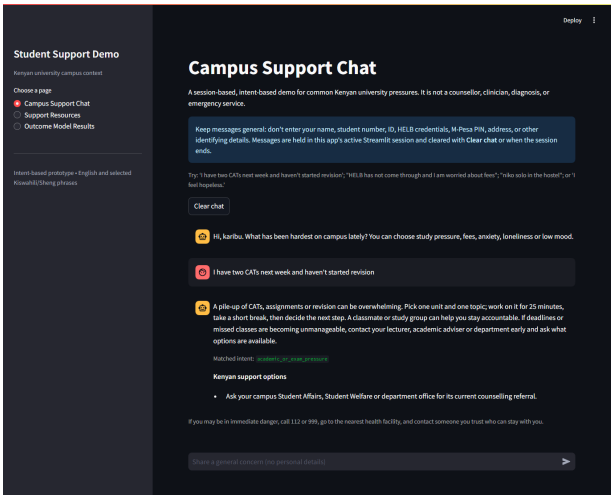
`PipelineConfig` is a frozen, slotted dataclass. It groups file paths, seed, split fraction, folds and permutation repeats. Frozen configuration helps prevent an unnoticed mid-run change. `DatasetParts`, `ModelResult` and `ValidationFinding` similarly give related values meaningful names. The project does not wrap each student row in a dataclass; DataFrames are more suitable for column operations and scikit-learn interoperability.

`FrameValidator` and `Predictor` are small Protocols. A Protocol specifies required methods structurally: an implementation can satisfy it without inheriting a base class. This permits a different validator or predictor while keeping the calling code simple. A direct function call would be a smaller alternative when only one implementation is expected.

`RunStage` is an Enum of fixed pipeline milestones. It demonstrates a bounded state vocabulary; pipeline logs still use the familiar stage names. This is useful when a value must come from a known set, but converting all dataset codes to Enums would add complexity without benefit.

The support component applies the same modelling choice to user language: `SupportIntent` is a `StrEnum` of bounded categories, and `SupportReply` is a frozen, slotted dataclass. This keeps the classifier's output explicit and testable. Intent matching is ordered so immediate-safety patterns take precedence over academic, financial or emotional categories.

Running interface: CAT/revision sample and matched academic-pressure intent



11. Advanced Python concepts II: resources and iteration

A context manager makes resource lifetime explicit. The HTTP response, ZIP archive and file handles are all entered using `with`, so they close on normal return and when an exception is raised. In contrast, a manual `open` followed by `close` can be skipped if an error interrupts the function; a `try/finally` block is the conventional lower-level alternative.

`iter_nonempty_lines` is a generator. Each iteration reads and yields one source line and its number, instead of constructing a second list containing every input line. The row-width check can therefore inspect large files with bounded extra memory. The test confirms blank lines are skipped while line numbering remains accurate.

This does not mean generators are always preferable. Once data is in a DataFrame, pandas vectorised operations efficiently inspect columns and nulls in compiled code. The project uses iteration for sequential file diagnostics and vectorized pandas checks for tabular properties.

`sha256_file` also streams fixed-size chunks. A deterministic content digest supports provenance checks and can reveal if a local copy differs. A checksum identifies exact bytes; it does not prove the source itself is authoritative, which is why the official URL and dataset ID are also documented.

The support app links to Kenyan services and official information: emergency services (112/999), LVCT Health's 1190 counselling information, Kenya Red Cross Chat Care and HELB student FAQs. The app does not invent a KCA-specific counselling number; it directs students to ask Student Affairs, Student Welfare, the Dean of Students or their department for the current referral pathway. Provider pages are the source of current availability.

12. Advanced Python concepts III: decorators and functional tools

`bounded_retry` is a decorator factory. It accepts retry settings and returns a decorator that wraps the function. The wrapper tries the fetch a bounded number of times, catches only expected network-related errors, logs retry attempts, and re-raises after the final failure. `functools.wraps` preserves the wrapped function name and documentation, which tests verify.

Bounded retries are useful because temporary connectivity failures may disappear after a short delay. An unlimited retry would hang indefinitely; retrying data or schema errors would not repair them. A simpler alternative is to place a short retry loop directly inside the download function, but a decorator makes the policy reusable and keeps the fetch implementation focused.

The implementation uses `ParamSpec` and `TypeVar` to preserve a wrapped function's call signature in type checking. `functools` and iterator utilities are used only where they provide a concrete purpose; other code remains ordinary, readable functions. Functional programming is not treated as a requirement to avoid mutation everywhere.

`_interpret_outcome` uses `match/case` to map target names to figure labels. Three explicit cases and a fallback make the allowed routes easy to scan. An `if/elif` chain would also be clear and is a valid simpler alternative.

The chatbot is intentionally deterministic: lowercased, whitespace-normalized input is checked against compiled regular expressions and the first match maps to a response. Patterns include Kenyan student terms such as CAT, revision, HELB, hostel, upkeep, "sina nauli", "niko solo", "nina wasiwasi" and "sijasoma". This demonstrates local examples without claiming broad Swahili or Sheng coverage. Unmatched messages return a general-support prompt.

13. Advanced Python concepts IV: diagnostics and exceptions

Python's `logging` module is used rather than `print` inside package functions. The CLI configures timestamped output; reusable modules log validation counts, model scores and a warning when a source header has whitespace. A caller can later change logging level or route logs to a file without changing analysis code.

The exception hierarchy starts with `PipelineError`. `DownloadError`, `InvalidArchiveError`, `SchemaError` and `ValidationError` let a command-line user distinguish failure causes. Low-level errors are chained with `raise ... from exc`, preserving diagnostic context while presenting domain language. The hierarchy is intentionally small; a separate class for every possible bad cell would not improve this coursework.

The frame validation contract is expressed as a Protocol, so a different implementation can be substituted if it offers the same method. The command line currently uses the concrete validator directly, which avoids unnecessary object construction. A Protocol is most valuable as the application grows or when tests substitute an implementation.

The design aims for explanatory clarity: structured errors and messages should help a student trace where a failure occurred. They do not replace tests, and catching overly broad exceptions would hide bugs. Network retries are narrowly scoped; parsing and validation failures stop the run.

The support responses avoid diagnosis and promise no human monitoring. Low mood prompts a qualified human referral; immediate-safety wording puts emergency support first. Financial guidance recommends contacting campus finance/student affairs and checking HELB through official channels; it warns students not to enter IDs, account credentials or M-Pesa PINs. These are signposting rules, not clinical triage. Regex matching can miss indirect language, misspelling or context, so the app displays immediate-safety guidance on every chat page as well as a dedicated crisis response.

14. Automated testing

Pytest tests cover extraction utilities, schema validation, transformations, estimator comparison and evaluation outputs. A shared fixture builds a miniature three-class 37-column frame. Tests run quickly and do not require downloading UCI because the large-data run is handled separately.

The tests include success cases and failure cases. Parameterised invalid-schema examples verify rejection of wrong column count and unexpected schema. Missing-value tests ensure the validator fails instead of inventing an imputation rule. The retry test checks both its attempt result and metadata preservation. The generator test checks lazy line numbering behaviour.

The model test exercises all three estimators with stratified CV on a small fixture. The evaluation test uses a dummy classifier only to verify the output-writing contract; it is not used for project performance claims. The actual reported model metrics come from the full official dataset pipeline execution.

Ruff provides linting and formatting checks; mypy checks package types. Pre-commit configuration offers consistent local formatting and lint checks. The new chatbot tests cover academic, financial, anxiety, loneliness, low-mood, greeting/thanks and immediate-safety routing, plus resource keys and the financial credential warning. The UI was opened locally and a CAT/revision sample produced the expected `academic_or_exam_pressure` label and advice. That interaction also caught and fixed a missing campus-support link mapping before final verification.

The complete suite now contains 20 passing tests. A verification run reported Ruff clean and mypy success across 12 source modules. Pytest emitted third-party deprecation warnings from installed pytest-asyncio and Matplotlib dependencies, not project test failures. UI checks are manual browser checks; there is no automated end-to-end browser suite, usability study, clinical validation or prospective evaluation.

15. Machine-learning protocol

The data is split once using stratification: 80% training (3,539 rows) and 20% testing (885 rows), with seed 4201. Stratification preserves approximate label proportions in each partition. The test partition is not used for model selection or fitting transformations.

Each candidate is a scikit-learn Pipeline with a ColumnTransformer followed by an estimator. During five-fold cross-validation on the training partition, each fold learns one-hot categories and numeric means/standard deviations from its own training fold. The fold validation data is transformed using those learned settings. This prevents preprocessing statistics from leaking across folds.

Logistic Regression is the linear baseline, configured with class weights balanced and maximum 2,000 iterations. Random Forest uses 300 trees, minimum leaf size two, and balanced class weights. Gradient Boosting uses 120 estimators, learning rate 0.08 and depth two. These modest settings avoid a large hyperparameter search. GradientBoostingClassifier does not support the same `class_weight` parameter, so no unsupported balancing option is forced onto it.

Macro-F1 is the primary comparison score; mean accuracy is secondary. Five-fold means and sample standard deviations are saved. The best mean CV macro-F1 chooses a model, which is refitted to all training data and assessed on the held-out test partition once.

16. Metrics and feature analysis

The final report includes overall macro-F1 and accuracy, per-class precision, recall and F1, and a confusion matrix. For a class, precision asks what fraction of predictions bearing that label were correct; recall asks what fraction of true examples of that class were found. F1 balances those two quantities.

Permutation importance is computed on the held-out partition using macro-F1 scoring and five shuffles per original feature. For each feature, the model score is measured, the feature values are shuffled, and the score drop is recorded. Mean and standard deviation are saved in a CSV and chart. This estimates dependence of this selected model on the feature under this data distribution.

Using the holdout for importance is an interpretive analysis after final model selection; it does not alter model selection or features. If importance results were used to tune the model, a new independent test set would be needed for an unbiased final claim. Correlated features can substitute for one another, and random shuffling can create unrealistic combinations.

An observed outcome classification should not be interpreted as a causal claim. Feature importance cannot establish that changing a feature will change a student's outcome. For real support decisions, class-specific error costs, calibrated probabilities, fairness, privacy, consent, and human review would need separate investigation.

17. Results: model comparison

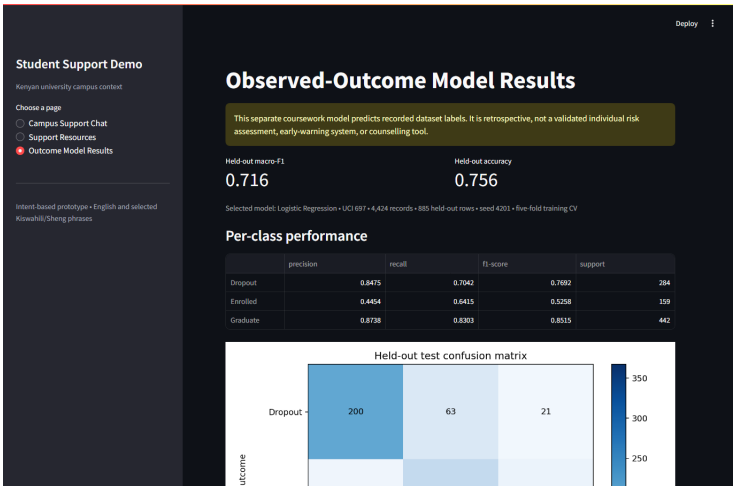
The three models were evaluated using the same seed, training partition, folds and macro-F1 selection rule. Logistic Regression attained training CV macro-F1 0.7052 (SD 0.0301) and accuracy 0.7457. Random Forest attained macro-F1 0.6998 (SD 0.0283) and accuracy 0.7627. Gradient Boosting attained macro-F1 0.6847 (SD 0.0327) and accuracy 0.7638.

Logistic Regression was selected because its mean cross-validation macro-F1 was highest. Although the two tree approaches had slightly higher cross-validation accuracy, their lower macro-F1 indicates a different balance across the three classes. This illustrates why the primary metric was chosen to reflect equal class importance.

On the untouched test partition, selected Logistic Regression achieved macro-F1 0.7155 and accuracy 0.7559. There were 885 test observations. The test score is close to the training CV mean but not a guarantee of performance in another sample or university. It is one split of one dataset, and its variability is not fully represented by a single test number.

The comparison CSV and CV JSON preserve the exact generated values. No metric in this report is an estimate copied from another study. The pipeline also records the random seed, split size, fold count, dataset digest and class counts in the run configuration.

Running interface: held-out results page (displayed values are project outputs)



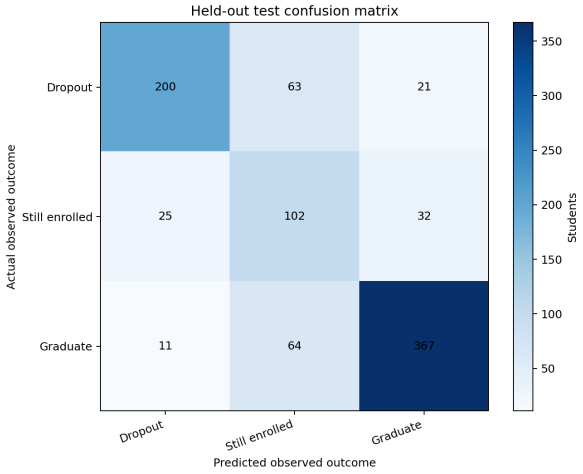
18. Results: class outcomes and importance

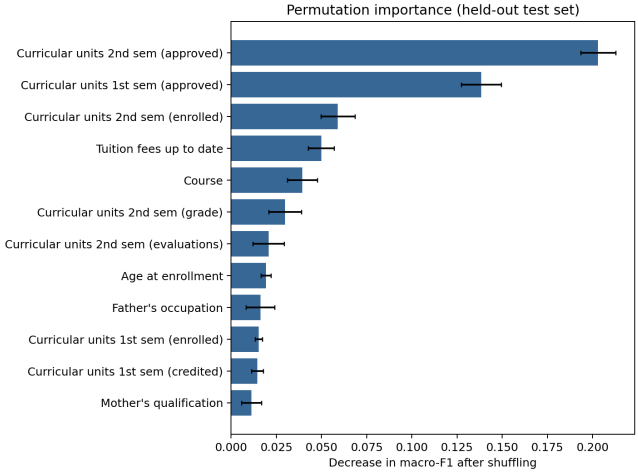
On the held-out set, Dropout precision was 0.847, recall 0.704 and F1 0.769 (284 true Dropout cases). Enrolled precision was 0.445, recall 0.642 and F1 0.526 (159 cases). Graduate precision was 0.874, recall 0.830 and F1 0.852 (442 cases). Enrolled is the weakest class by F1; many predictions made as Enrolled were actually another outcome.

The confusion matrix rows are actual classes and columns predicted classes. Of 284 Dropout cases, 200 were correctly labelled, 63 predicted Enrolled and 21 Graduate. Of 159 Enrolled cases, 102 were correct, 25 predicted Dropout and 32 Graduate. Of 442 Graduates, 367 were correct, 64 predicted Enrolled and 11 Dropout. These errors matter differently depending on how a result might be used.

Second-semester approved units had mean permutation macro-F1 decrease 0.203; first-semester approved units had 0.138. Both are substantially above the other predictors in this fitted model. They are indicators of observed academic progress, not causes of graduation or dropout. Their timing may also be late for an early intervention.

The figures below and the CSV artifacts are generated by the pipeline from actual execution.





19. Kenyan campus adaptation, reproducibility and limitations

Kenyan campus examples include CATs, revision backlogs, fee balances, HELB/HEF, bursary referrals, rent/hostel costs, transport fare, upkeep, anxiety, loneliness and low mood. The advice favours realistic next steps: a short study block and peer/lecturer support; campus finance/student-affairs referral; grounding plus human help for persistent worry; and manageable social contact. Messages remain general and active-session-only, with a clear-chat control. No student identifiers should be submitted.

Resources link to official or provider information for emergency services (112/999), LVCT Health's 1190 counselling line, Kenya Red Cross Chat Care and HELB student FAQs. Campus-specific guidance says to ask Student Affairs, Student Welfare, the Dean of Students or a department for the current referral. The project does not assert a verified KCA counselling telephone number. The app is not monitored; it directs immediate danger toward emergency help and a trusted person, not a delayed chatbot response.

The package specifies Python 3.11+, dependencies and CLI/app entry commands. A fixed random seed, configuration JSON and dataset SHA-256 make the experiment traceable. The full suite passed 20 tests; Ruff and mypy checks passed on the corrected source. The manually inspected Streamlit session verified the chat flow and localized intent response. There has been no clinical validation, student usability study or automated browser test.

The outcome model is limited by a single source cohort, random rather than temporal split, class imbalance, feature dependence and no subgroup evaluation. Second-semester features may be close to the recorded outcome. It is not a validated early-warning system. The chatbot has limited regular-expression vocabulary, does not interpret context reliably and cannot provide clinical assessment. A future deployment would need local student and counsellor co-design, approved referral contacts, privacy/security review, accessibility and language evaluation, escalation ownership, consent, and evaluation with qualified professionals. Neither component should decide eligibility, discipline, funding or clinical care.

20. Conclusion and references

The project now combines a reproducible academic-outcome pipeline with a separate Kenyan campus support prototype. The pipeline downloads and validates the public UCI dataset, keeps preprocessing inside cross-validated pipelines, compares three multiclass models and writes reproducible artifacts. Logistic Regression achieved held-out macro-F1 0.716 and accuracy 0.756; Enrolled remained its weakest class. Academic progress features dominate the permutation ranking, but their timing rules out an early-warning claim.

The campus app demonstrates deterministic intent routing for exam pressure, money stress, anxiety, low mood, loneliness and immediate safety, with Kenyan resource links and explicit limits. It does not connect personal text to prediction results or diagnose a student. Key viva points are cross-validation leakage, macro-F1 under class imbalance, categorical encoding, predictive versus causal importance, regex intent order, resource verification and the need for human-owned escalation. The work is a coursework prototype, not an operational intervention.

References

1. Realinho, V., Machado, J., Baptista, L., & Martins, M. V. (2022). Predicting Student Dropout and Academic Success. *Data, 7*(11), 146. https://doi.org/10.3390/data7110146
2. Realinho, V., Vieira Martins, M., Machado, J., & Baptista, L. (2021). *Predict Students' Dropout and Academic Success* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5MC89
3. Martins, M. V., Baptista, L., Machado, J., & Realinho, V. (2023). Multi-Class Phased Prediction of Academic Performance and Dropout in Higher Education. *Applied Sciences, 13*(8), 4702. https://doi.org/10.3390/app13084702
4. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.
5. Scikit-learn developers. (2026). Pipeline and composite estimators; cross-validation; permutation feature importance documentation. https://scikit-learn.org/stable/
6. Python Software Foundation. (2026). Python 3.11 documentation: Data model, typing, dataclasses, context managers and functional programming. https://docs.python.org/3.11/
7. pytest developers. (2026). pytest documentation. https://docs.pytest.org/
8. National Police Service, Kenya. Emergency services and police contacts. https://www.nationalpolice.go.ke/
9. LVCT Health. Counselling and psychosocial support information for 1190. https://lvcthealth.org/a-counselor-experience-in-saving-a-suicide-mind/
10. Kenya Red Cross Society. Chat Care. https://redcross.or.ke/chat-care/
11. Higher Education Loans Board, Kenya. Student FAQs. https://www.helb.co.ke/faqs/students-faqs/

The Kenyan provider links were checked on 21 October 2026. Service availability and details may change; users should check the linked provider for current information.